

=====

CAPSTONE PROJECT

NOTEBOOK 25: MONGODB INGESTION FOR MULTI-LABEL PHASE

=====

Purpose:

This notebook loads the prepared FMA
multi-label metadata

into MongoDB so it can be used as the data
engineering layer

for full metadata genre identification.

The goal is to:

1. Load genre inventory and multi-label
master tables
2. Build MongoDB-ready genre and track
documents
3. Insert documents into MongoDB
collections

4. Create useful indexes for retrieval and modelling

5. Validate that the database is ready for the next phase

=====

In [1]:

```
# =====
# 1. IMPORT LIBRARIES
# =====

import os
import math
import pandas as pd
from pymongo import MongoClient, ASCENDING
```

In [2]:

```
# =====
# 2. LOAD PREPARED CSV FILES
# =====

genre_inventory = pd.read_csv("../data/processed/full_genre_inventory.csv")
candidate_genres = pd.read_csv("../data/processed/modelling_candidate_genres.csv")
master_df = pd.read_csv("../data/processed/large_multilabel_master_table.csv")

print("Genre inventory shape:", genre_inventory.shape)
print("Candidate genres shape:", candidate_genres.shape)
print("Master table shape:", master_df.shape)

display(genre_inventory.head())
display(candidate_genres.head())
display(master_df.head())
```

```
Genre inventory shape: (163, 11)
Candidate genres shape: (150, 11)
Master table shape: (81574, 10)
```

	genre_id	genre_name	parent_id	parent_name	root_genre_id	root_genre_name	top_level_1
0	38	Experimental	NaN	NaN	38	Experimental	
1	15	Electronic	NaN	NaN	15	Electronic	
2	12	Rock	NaN	NaN	12	Rock	
3	1235	Instrumental	NaN	NaN	1235	Instrumental	1
4	10	Pop	NaN	NaN	10	Pop	



	genre_id	genre_name	parent_id	parent_name	root_genre_id	root_genre_name	top_level_1
0	38	Experimental	NaN	NaN	38	Experimental	
1	15	Electronic	NaN	NaN	15	Electronic	
2	12	Rock	NaN	NaN	12	Rock	
3	1235	Instrumental	NaN	NaN	1235	Instrumental	1
4	10	Pop	NaN	NaN	10	Pop	



	track_id	split	subset	genre_top	title	genres_ids	genres_all_ids	genres_all_names
0	20	training	large	NaN	Spiritual Level	76,103	17,10,76,103	Folk Pop Experimental Pop Singer-Songwriter
1	26	training	large	NaN	Where is your Love?	76,103	17,10,76,103	Folk Pop Experimental Pop Singer-Songwriter
2	30	training	large	NaN	Too Happy	76,103	17,10,76,103	Folk Pop Experimental Pop Singer-Songwriter
3	46	training	large	NaN	Yosemite	76,103	17,10,76,103	Folk Pop Experimental Pop Singer-Songwriter
4	48	training	large	NaN	Light of Light	76,103	17,10,76,103	Folk Pop Experimental Pop Singer-Songwriter



In [3]:

```
# =====  
# 3. DEFINE HELPER FUNCTIONS
```

```
# =====

def parse_csv_id_list(value):
    if pd.isna(value):
        return []
    text = str(value).strip()
    if text == "":
        return []
    return [int(x) for x in text.split(",") if str(x).strip() != ""]

def parse_name_list(value):
    if pd.isna(value):
        return []
    text = str(value).strip()
    if text == "":
        return []
    return [x.strip() for x in text.split("|") if x.strip() != ""]

def clean_nan(value):
    if pd.isna(value):
        return None
    return value

def to_int_or_none(value):
    if pd.isna(value):
        return None
    try:
        return int(value)
    except:
        return None
```

In [4]:

```
# =====
# 4. PREPARE LOOKUP SETS AND MAPS
# =====

candidate_label_ids = set(candidate_genres["genre_id"].astype(int).tolist())
all_used_label_ids = set()

master_df["genres_ids_list"] = master_df["genres_ids"].apply(parse_csv_id_list)
master_df["genres_all_ids_list"] =
master_df["genres_all_ids"].apply(parse_csv_id_list)
master_df["genres_all_names_list"] =
master_df["genres_all_names"].apply(parse_name_list)

for ids in master_df["genres_all_ids_list"]:
    all_used_label_ids.update(ids)

all_used_label_ids = set(sorted(all_used_label_ids))
rare_tail_label_ids = set(sorted(all_used_label_ids - candidate_label_ids))

genre_name_map = dict(zip(
    genre_inventory["genre_id"].astype(int),
    genre_inventory["genre_name"].astype(str)
```

```

))

root_genre_name_map = dict(zip(
    genre_inventory["genre_id"].astype(int),
    genre_inventory["root_genre_name"].astype(str)
))

print("Used full labels:", len(all_used_label_ids))
print("Candidate labels:", len(candidate_label_ids))
print("Rare-tail labels:", len(rare_tail_label_ids))

```

Used full labels: 161
Candidate labels: 150
Rare-tail labels: 11

In [5]:

```

# =====
# 5. BUILD GENRE LOOKUP DOCUMENTS
# =====

genres_lookup_docs = []

for _, row in genre_inventory.iterrows():
    genre_id = int(row["genre_id"])

    doc = {
        "_id": genre_id,
        "genre_id": genre_id,
        "genre_name": str(row["genre_name"]),
        "parent_id": to_int_or_none(row.get("parent_id")),
        "parent_name": clean_nan(row.get("parent_name")),
        "root_genre_id": to_int_or_none(row.get("root_genre_id")),
        "root_genre_name": clean_nan(row.get("root_genre_name")),
        "top_level_flag": to_int_or_none(row.get("top_level_flag")),
        "full_direct_count": int(row.get("full_direct_count", 0)),
        "full_genres_all_count": int(row.get("full_genres_all_count", 0)),
        "large_audio_genres_all_count": int(row.get("large_audio_genres_all_count",
0)),
        "modelling_tier": str(row.get("modelling_tier", "")),
        "used_in_metadata": genre_id in all_used_label_ids,
        "candidate_label": genre_id in candidate_label_ids,
        "rare_tail_label": genre_id in rare_tail_label_ids
    }

    genres_lookup_docs.append(doc)

print("Genre lookup documents:", len(genres_lookup_docs))
print(genres_lookup_docs[:3])

```

Genre lookup documents: 163

```
[{'_id': 38, 'genre_id': 38, 'genre_name': 'Experimental', 'parent_id': None,
'parent_name': None, 'root_genre_id': 38, 'root_genre_name': 'Experimental',
'top_level_flag': 38, 'full_direct_count': 24912, 'full_genres_all_count': 38154,
'large_audio_genres_all_count': 35903, 'modelling_tier': 'Tier 1: Strong',
'used_in_metadata': True, 'candidate_label': True, 'rare_tail_label': False}, {'_id':
15, 'genre_id': 15, 'genre_name': 'Electronic', 'parent_id': None, 'parent_name':
None, 'root_genre_id': 15, 'root_genre_name': 'Electronic', 'top_level_flag': 15,
'full_direct_count': 23866, 'full_genres_all_count': 34413,
'large_audio_genres_all_count': 28099, 'modelling_tier': 'Tier 1: Strong',
'used_in_metadata': True, 'candidate_label': True, 'rare_tail_label': False}, {'_id':
12, 'genre_id': 12, 'genre_name': 'Rock', 'parent_id': None, 'parent_name': None,
'root_genre_id': 12, 'root_genre_name': 'Rock', 'top_level_flag': 12,
'full_direct_count': 8038, 'full_genres_all_count': 32923,
'large_audio_genres_all_count': 25820, 'modelling_tier': 'Tier 1: Strong',
'used_in_metadata': True, 'candidate_label': True, 'rare_tail_label': False}]
```

In [6]:

```
# =====
# 6. BUILD TRACK MULTI-LABEL DOCUMENTS
# =====

track_docs = []

for _, row in master_df.iterrows():
    genres_ids_list = row["genres_ids_list"]
    genres_all_ids_list = row["genres_all_ids_list"]
    genres_all_names_list = row["genres_all_names_list"]

    candidate_genres_all_ids = [gid for gid in genres_all_ids_list if gid in
candidate_label_ids]
    rare_tail_genres_all_ids = [gid for gid in genres_all_ids_list if gid in
rare_tail_label_ids]

    candidate_genres_all_names = [genre_name_map.get(gid, f"genre_{gid}") for gid
in candidate_genres_all_ids]
    rare_tail_genres_all_names = [genre_name_map.get(gid, f"genre_{gid}") for gid
in rare_tail_genres_all_ids]

    root_genre_names = sorted(list({
        root_genre_name_map.get(gid, None)
        for gid in genres_all_ids_list
        if root_genre_name_map.get(gid, None) is not None
    }))

    doc = {
        "_id": int(row["track_id"]),
        "track_id": int(row["track_id"]),
        "split": str(row["split"]),
        "subset": str(row["subset"]),
        "genre_top": clean_nan(row["genre_top"]),
        "title": clean_nan(row["title"]),
        "audio_path": str(row["audio_path"]),
        "audio_exists": bool(row["audio_exists"]),

        "genres_ids": genres_ids_list,
```

```

"genres_all_ids": genres_all_ids_list,
"genres_all_names": genres_all_names_list,

"candidate_genres_all_ids": candidate_genres_all_ids,
"candidate_genres_all_names": candidate_genres_all_names,
"rare_tail_genres_all_ids": rare_tail_genres_all_ids,
"rare_tail_genres_all_names": rare_tail_genres_all_names,

"label_counts": {
    "genres_ids_count": len(genres_ids_list),
    "genres_all_count": len(genres_all_ids_list),
    "candidate_label_count": len(candidate_genres_all_ids),
    "rare_tail_label_count": len(rare_tail_genres_all_ids)
},

"root_genre_names": root_genre_names,

"modelling_flags": {
    "usable_for_candidate_multilabel": len(candidate_genres_all_ids) > 0,
    "has_rare_tail_labels": len(rare_tail_genres_all_ids) > 0
}
}

track_docs.append(doc)

print("Track documents:", len(track_docs))
print(track_docs[:2])

```

Track documents: 81574

```

[{'_id': 20, 'track_id': 20, 'split': 'training', 'subset': 'large', 'genre_top':
None, 'title': 'Spiritual Level', 'audio_path':
'../data/raw/audio/fma_large\\000\\000020.mp3', 'audio_exists': True, 'genres_ids':
[76, 103], 'genres_all_ids': [17, 10, 76, 103], 'genres_all_names': ['Folk', 'Pop',
'Experimental Pop', 'Singer-Songwriter'], 'candidate_genres_all_ids': [17, 10, 76,
103], 'candidate_genres_all_names': ['Folk', 'Pop', 'Experimental Pop', 'Singer-
Songwriter'], 'rare_tail_genres_all_ids': [], 'rare_tail_genres_all_names': [],
'label_counts': {'genres_ids_count': 2, 'genres_all_count': 4,
'candidate_label_count': 4, 'rare_tail_label_count': 0}, 'root_genre_names': ['Folk',
'Pop'], 'modelling_flags': {'usable_for_candidate_multilabel': True,
'has_rare_tail_labels': False}}, {'_id': 26, 'track_id': 26, 'split': 'training',
'subset': 'large', 'genre_top': None, 'title': 'Where is your Love?', 'audio_path':
'../data/raw/audio/fma_large\\000\\000026.mp3', 'audio_exists': True, 'genres_ids':
[76, 103], 'genres_all_ids': [17, 10, 76, 103], 'genres_all_names': ['Folk', 'Pop',
'Experimental Pop', 'Singer-Songwriter'], 'candidate_genres_all_ids': [17, 10, 76,
103], 'candidate_genres_all_names': ['Folk', 'Pop', 'Experimental Pop', 'Singer-
Songwriter'], 'rare_tail_genres_all_ids': [], 'rare_tail_genres_all_names': [],
'label_counts': {'genres_ids_count': 2, 'genres_all_count': 4,
'candidate_label_count': 4, 'rare_tail_label_count': 0}, 'root_genre_names': ['Folk',
'Pop'], 'modelling_flags': {'usable_for_candidate_multilabel': True,
'has_rare_tail_labels': False}}]

```

In [7]:

```

# =====
# 7. CONNECT TO MONGODB
# =====

```

```

MONGO_URI = "mongodb://localhost:27017/"
DB_NAME = "fma_capstone"

client = MongoClient(MONGO_URI)
db = client[DB_NAME]

print("Connected to MongoDB.")
print("Database name:", DB_NAME)

```

Connected to MongoDB.
Database name: fma_capstone

In [8]:

```

# =====
# 8. CREATE / RESET COLLECTIONS
# =====

db.drop_collection("genres_lookup")
db.drop_collection("tracks_multilabel")

genres_collection = db["genres_lookup"]
tracks_collection = db["tracks_multilabel"]

print("Dropped old collections if they existed.")

```

Dropped old collections if they existed.

In [9]:

```

# =====
# 9. INSERT GENRE LOOKUP DOCUMENTS
# =====

if len(genres_lookup_docs) > 0:
    genres_collection.insert_many(genres_lookup_docs)

print("Inserted genre lookup documents:", genres_collection.count_documents({}))

```

Inserted genre lookup documents: 163

In [10]:

```

# =====
# 10. INSERT TRACK DOCUMENTS IN BATCHES
# =====

BATCH_SIZE = 5000

for start in range(0, len(track_docs), BATCH_SIZE):
    end = min(start + BATCH_SIZE, len(track_docs))
    batch = track_docs[start:end]
    tracks_collection.insert_many(batch)

```



```
print(f"Inserted track docs {start} to {end}")

print("Inserted track documents:", tracks_collection.count_documents({}))
```

```
Inserted track docs 0 to 5000
Inserted track docs 5000 to 10000
Inserted track docs 10000 to 15000
Inserted track docs 15000 to 20000
Inserted track docs 20000 to 25000
Inserted track docs 25000 to 30000
Inserted track docs 30000 to 35000
Inserted track docs 35000 to 40000
Inserted track docs 40000 to 45000
Inserted track docs 45000 to 50000
Inserted track docs 50000 to 55000
Inserted track docs 55000 to 60000
Inserted track docs 60000 to 65000
Inserted track docs 65000 to 70000
Inserted track docs 70000 to 75000
Inserted track docs 75000 to 80000
Inserted track docs 80000 to 81574
Inserted track documents: 81574
```

In [11]:

```
# =====
# 11. CREATE INDEXES
# =====

genres_collection.create_index([("genre_id", ASCENDING)], unique=True)
genres_collection.create_index([("genre_name", ASCENDING)])
genres_collection.create_index([("root_genre_id", ASCENDING)])
genres_collection.create_index([("candidate_label", ASCENDING)])
genres_collection.create_index([("rare_tail_label", ASCENDING)])

tracks_collection.create_index([("track_id", ASCENDING)], unique=True)
tracks_collection.create_index([("split", ASCENDING)])
tracks_collection.create_index([("subset", ASCENDING)])
tracks_collection.create_index([("genre_top", ASCENDING)])
tracks_collection.create_index([("audio_exists", ASCENDING)])
tracks_collection.create_index([("genres_all_ids", ASCENDING)])
tracks_collection.create_index([("candidate_genres_all_ids", ASCENDING)])
tracks_collection.create_index([("rare_tail_genres_all_ids", ASCENDING)])
tracks_collection.create_index([("root_genre_names", ASCENDING)])
tracks_collection.create_index([("modelling_flags.usable_for_candidate_multilabel",
ASCENDING)])

print("Indexes created.")
```

Indexes created.

In [12]:

```
# =====
# 12. RUN SANITY CHECK QUERIES
```

```
# =====

print("Total genres in MongoDB:", genres_collection.count_documents({}))
print("Total tracks in MongoDB:", tracks_collection.count_documents({}))

print("\nCandidate-label genres in MongoDB:")
print(genres_collection.count_documents({"candidate_label": True}))

print("\nRare-tail genres in MongoDB:")
print(genres_collection.count_documents({"rare_tail_label": True}))

print("\nTracks usable for candidate multi-label modelling:")
print(tracks_collection.count_documents({"modelling_flags.usable_for_candidate_multilabel": True}))

print("\nExample track document:")
example_doc = tracks_collection.find_one({}, {"_id": 1, "track_id": 1, "split": 1, "genre_top": 1, "genres_all_names": 1, "candidate_genres_all_names": 1, "rare_tail_genres_all_names": 1, "root_genre_names": 1})
print(example_doc)

print("\nExample genre document:")
example_genre_doc = genres_collection.find_one({}, {"_id": 1, "genre_id": 1, "genre_name": 1, "root_genre_name": 1, "candidate_label": 1, "rare_tail_label": 1})
print(example_genre_doc)
```

Total genres in MongoDB: 163

Total tracks in MongoDB: 81574

Candidate-label genres in MongoDB:

150

Rare-tail genres in MongoDB:

11

Tracks usable for candidate multi-label modelling:

79343

Example track document:

```
{'_id': 20, 'track_id': 20, 'split': 'training', 'genre_top': None, 'genres_all_names': ['Folk', 'Pop', 'Experimental Pop', 'Singer-Songwriter'], 'candidate_genres_all_names': ['Folk', 'Pop', 'Experimental Pop', 'Singer-Songwriter'], 'rare_tail_genres_all_names': [], 'root_genre_names': ['Folk', 'Pop']}
```

Example genre document:

```
{'_id': 38, 'genre_id': 38, 'genre_name': 'Experimental', 'root_genre_name': 'Experimental', 'candidate_label': True, 'rare_tail_label': False}
```

In [13]:

```
# =====
# 13. SAVE COLLECTION SUMMARY
# =====

collection_summary = pd.DataFrame([
    {
```

```

        "collection": "genres_lookup",
        "document_count": genres_collection.count_documents({}),
        "description": "Genre inventory, hierarchy, candidate flags, rare-tail
flags"
    },
    {
        "collection": "tracks_multilabel",
        "document_count": tracks_collection.count_documents({}),
        "description": "Unified multi-label track records for full metadata genre
identification"
    }
])

os.makedirs("../data/processed", exist_ok=True)

collection_summary.to_csv(
    "../data/processed/mongodb_collection_summary.csv",
    index=False
)

print("Saved MongoDB collection summary.")
display(collection_summary)

```

Saved MongoDB collection summary.

	collection	document_count	description
0	genres_lookup	163	Genre inventory, hierarchy, candidate flags, r...
1	tracks_multilabel	81574	Unified multi-label track records for full met...

In [14]:

```

# =====
# 14. INTERPRETATION NOTES
# =====

print("1. MongoDB is now set up as the metadata and label management layer for the
multi-label phase.")
print("2. The genres_lookup collection stores hierarchy and label usability
information.")
print("3. The tracks_multilabel collection stores one document per large-audio
track with multi-label genre arrays.")
print("4. This database layer can now be queried to generate modelling subsets for
structured, audio, and hybrid multi-label experiments.")
print("5. The next modelling step should be a structured multi-label baseline on
the 150 candidate labels.")

```

1. MongoDB is now set up as the metadata and label management layer for the multi-label phase.
2. The `genres_lookup` collection stores hierarchy and label usability information.
3. The `tracks_multilabel` collection stores one document per large-audio track with multi-label genre arrays.
4. This database layer can now be queried to generate modelling subsets for structured, audio, and hybrid multi-label experiments.
5. The next modelling step should be a structured multi-label baseline on the 150 candidate labels.